Part 11 — From model to design: strain-design algorithms, machine learning, labelling and the bioreactor (weeks 39 to 44)

These six weeks are where the modelling of part 10 becomes a design tool and where the transport arithmetic of part 3 comes back to constrain it. The core model built by hand in part 10 is reused throughout: it supplies the production envelopes and the knockout search in chapter 39, the proteome-constrained overflow in the same chapter, and — in chapter 44 — the oxygen exchange bound that a k_La calculation finally fixes, which is the point at which the transport calculation and the genome-scale model meet.

Almost everything quantitative here was run. The strain-design search, the Bayesian optimisation against a factorial grid, the overfitting and leakage simulations, the ^{13}C fitting and the oxygen-bound sweep are all executed code with the parameters stated. The exceptions, marked where they occur, are the retrosynthesis tools in chapter 40 and the flux-fitting packages in chapter 43, which need network access or software this environment does not have; for those the chapters give the arithmetic and the reading rather than output.

Two results worth flagging before you start. The knockout search in chapter 39 found no growth-coupled design at all under aerobic conditions and plenty under anaerobic — which is not a bug but the reason real succinate strains are anaerobic, and it only became obvious because the search failed first. And the Bayesian optimisation in chapter 41 beat a 16-run factorial grid decisively in the median and still got stuck on a local optimum in one seed out of forty, which is the honest version of a method that is usually sold without its failure mode.

Ch.	Week	Chapter
39	39	Strain-design algorithms: growth coupling, searched by hand
40	40	Retrosynthesis and pathway prediction: expanding a network backwards
41	41	Bayesian optimisation: choosing the next experiment from a dozen noisy points
42	42	Machine learning for biology: overfitting, leakage and the split that matches the question
43	43	^{13}C metabolic flux analysis: what labelling resolves that stoichiometry cannot
44	44	Bioreactor engineering and scale-up: mixing, transfer, feeding and the cost sketch

Strain-design algorithms: growth coupling, searched by hand

Growth coupling means arranging the network so that the cell cannot grow without making your product. Then evolution works for you instead of against you — which is the only way to win against a population that has 10^9 members and a generation time of twenty minutes.

OptKnock, and the bilevel structure

Burgard, Pharkya and Maranas posed strain design as a **bilevel optimisation**. The outer problem chooses which reactions to delete, subject to a limit on how many; the inner problem is the cell's own — given those deletions, it distributes flux to maximise growth. The outer objective is the product flux *at the inner problem's optimum*. In words: find the deletions such that, when the cell does the selfish thing, the product comes out anyway.

Growth coupling in one sentence for a non-specialist: *we break the metabolic routes the cell would otherwise use to balance its books, so that the only remaining way for it to grow is to make and excrete our product.* The reason it matters is the one from part 1 chapter 10: a strain that makes a product at a cost to growth is losing a race against its own revertants, whereas a growth-coupled strain is winning it, because a mutant that stops producing also stops growing.

Searching the design space by enumeration

OptKnock solves the bilevel problem as a mixed-integer programme. With a 40-reaction model, you can do something cruder and more instructive: enumerate. For every candidate deletion set, solve the inner problem for maximum growth, then minimise the product flux at that growth — the **guaranteed** product, the amount the cell must make even in its best case for itself. That minimisation is the step that distinguishes coupling from possibility, and it is what the production envelope shows graphically.

```
def coupled(ko, prod='EX_succ', frac=0.2):
    l, u = ANA.copy(), ub.copy()                # anaerobic base case
    for k in ko: l[idx[k]] = 0; u[idx[k]] = 0    # the deletions
    g = opt(idx['BIOMASS'], l, u)               # inner problem: the cell maximises growth
    if g is None or g < frac*gmax: return None   # reject designs that barely grow
    return opt(idx[prod], l, u, maximize=False, fix={'BIOMASS': g}), g   # guaranteed product
```

The first run found nothing. Searching single and double deletions aerobically returned no design with any guaranteed succinate: with oxygen available the cell can always oxidise its substrate completely and balance its redox books without the product. That failure is the result — it is why real succinate strains are anaerobic or micro-aerobic, and it is a cleaner demonstration of the principle than any successful design would have been. Rerunning anaerobically:

Deletions	Guaranteed succinate (mmol gDW⁻¹ h⁻¹)	Growth (h⁻¹)
none (wild type)	0	0.217
ΔPYK (pyruvate kinase)	8.23	0.134
ΔPFL	2.94	0.111
ΔADH	1.98	0.123
ΔACK (acetate kinase)	1.75	0.155
ΔPYK ΔACK	9.05	0.072
ΔPYK ΔACK ΔPFL	9.36	0.048

Four of twenty-seven single deletions couple at all; fifty-two of the pairs do. The pattern is the trade every strain designer meets: **more coupling costs more growth**. ΔPYK gives 8.23 at 0.134 h^{-1}; adding ΔACK buys 10% more product and halves the growth rate; the triple buys 3% more and halves it again. Which point on that curve you want depends on the process — a slow strain needs a longer fermentation and a bigger seed train — and the model cannot tell you, because the choice is economic.

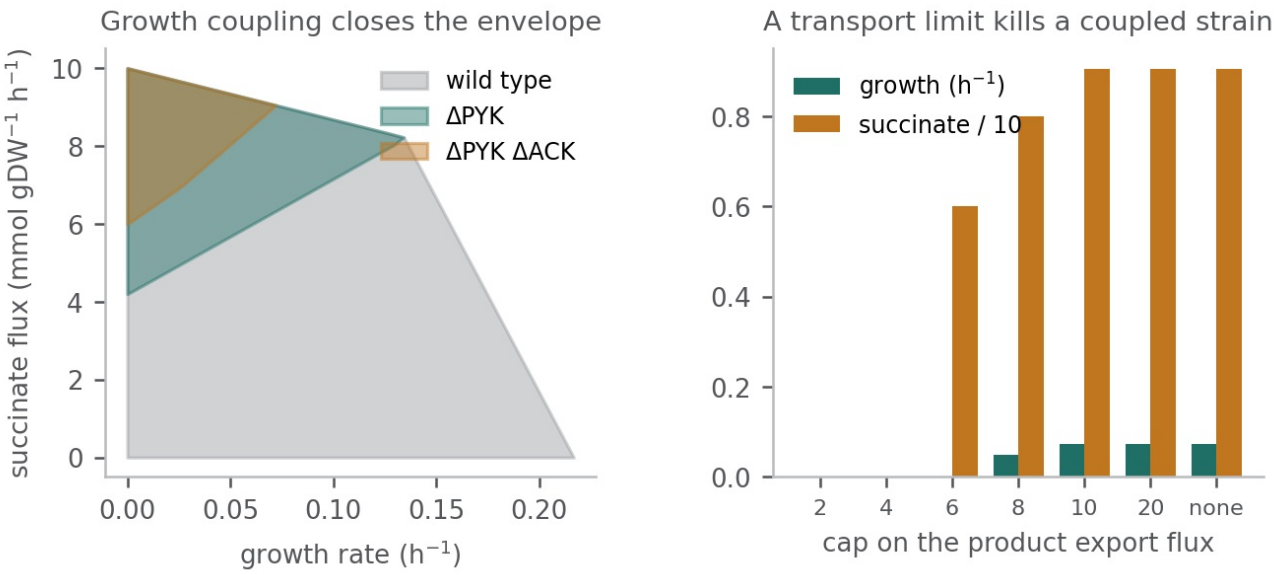


Figure 39.1 Left: production envelopes — the range of succinate flux possible at each growth rate — for the wild type and two knockout strains, anaerobically. Coupling is the closing of the envelope's lower edge: at every growth rate the strain must make product. Right: what happens when the product's export flux is capped. Computed with the hand-built core model of part 10.

Comparing with the literature, as the week asks: the published anaerobic succinate strains delete the competing fermentative routes — lactate dehydrogenase, pyruvate formate-lyase, acetate kinase and phosphotransacetylase — which my search also found, and they additionally engineer the carboxylation step and the glucose uptake system, which my model cannot suggest because its PEP-consuming phosphotransferase system is the only uptake route it has. That is the honest difference: the model agrees on the deletions it can see and is silent about the interventions outside its scope. What the experiment found that the model missed is usually of this kind — a transporter, a regulator, or an evolved tolerance.

The export bound: a transport limit made numerical

The right-hand panel is the week's most useful exercise. Take the ΔPYK ΔACK strain and cap the succinate export flux, as a real transporter's V_{max} would:

Export cap	Growth	Guaranteed product
none	0.072	9.05
10	0.072	9.05
6	0.000	6.00
4 and below	infeasible — no solution at all	

Below about 6 the strain cannot grow, and below 4 the model has no feasible steady state: the deletions have made product formation obligatory, so a cell that cannot export its product cannot balance its internal pools and dies. **Growth coupling and a transport limit interact catastrophically.** That is the quantitative version of part 3 chapter 45's argument, and it is the reason an export measurement belongs in a design campaign before the knockouts, not after.

What the proteome constraint adds: overflow, from first principles

Part 4 chapter 51 claimed that adding a proteome budget makes a constraint-based model predict overflow metabolism instead of needing to be told about it. I tested that on the core model by assigning each reaction a crude enzyme cost per unit flux — respiration and the TCA cycle expensive, fermentative routes cheap, exchanges free — and constraining the total. The costs are illustrative, not measured.

Glucose uptake	No proteome constraint		Budget fixed at the cost of the solution at 10			
	growth	acetate	growth	acetate	ethanol	O_2 uptake
10	0.754	0	0.754	0	0	9.5
12	0.906	0	0.827	0.18	0.82	9.6
15	1.131	0	0.777	0.75	7.20	7.4

20	1.508	0	0.673	4.34	16.6	3.1

Without the constraint, growth rises linearly and nothing is ever excreted. With it, by-products appear above a threshold uptake and **oxygen uptake falls as the uptake rate rises** — the cell switching from protein-expensive respiration to protein-cheap fermentation, which is exactly the proteome-allocation account of overflow from part 1 chapter 6. The growth decline beyond 12 is an artefact of my hard budget with no maintenance scaling, so take the onset as the result and not the shape at high uptake.

^{13}C-MFA, and the three kinds of flux

The distinction the week wants, which chapter 43 develops: an **FBA-feasible** flux is consistent with stoichiometry and an assumed objective — a hypothesis. A flux **constrained by measured exchange rates** is pinned at the boundary and still undetermined inside, as part 10's flux variability showed, where fourteen reactions had ranges at exactly optimal growth. A **^{13}C-measured** flux is inferred from labelling patterns and resolves the internal splits that neither of the others can: the division at glucose-6-phosphate, the reversibility of exchange fluxes, and anaplerotic against cyclic TCA flux.

What kinetic and enzyme-constrained models add, and cost

Model class	Adds	Costs
FBA	Yields, feasibility, deletion predictions	An objective assumption; no concentrations, no time
Enzyme-constrained (GECKO-style)	Overflow, growth-rate-dependent proteome allocation, a protein cost per flux	A k_{cat} and a molecular weight per reaction — and literature k_{cat} values are frequently wrong by an order of magnitude (part 4 chapter 51)
Kinetic	Transients, pool concentrations, control coefficients, regulatory behaviour	Three to five parameters per reaction, most unidentifiable from available data (part 2 chapter 16)

Bilevel	`outer: choose deletions · inner: cell maximises growth · objective: product at the inner optimum`
Guaranteed product	`min(product) subject to growth = max growth — the lower edge of the envelope`
Coupling trade	`ΔPYK 8.23 at μ 0.134; ΔPYK ΔACK 9.05 at 0.072; triple 9.36 at 0.048`
Export cap	`below 6 the coupled strain cannot grow; below 4 no feasible solution exists`
Proteome budget	`Σ (cost × flux) ≤ P reproduces overflow and falling O₂ uptake`

Your OptKnock design predicts a coupled yield of 70% of theoretical with four deletions. What do you check before building it?

Four things, in order of how cheaply they can kill the design. **The oxygen condition**: my search found nothing aerobically and plenty anaerobically, so check which the design assumes and whether the process can be run that way — an anaerobic design in an aerobic process is not a design. **The export flux**: cap the product exchange at a realistic transporter V_{max} and rerun, because the table above shows a coupled strain dying when export is limited; if the yield collapses, the transporter is the first thing to engineer. **The growth rate**: 70% of theoretical at what μ? A design at 0.05 h^{-1} is a fourteen-hour doubling time and a long, contamination-prone fermentation. **Alternative optima**: run flux variability at the predicted optimum to check the product flux is genuinely forced and not merely possible.

Then the things the model cannot see, which decide whether four deletions become a strain: whether each deleted gene has an isozyme that the gene–protein–reaction rules missed, whether the quadruple mutant is viable at all on the medium you will use, and whether the strain evolves its way out — which, for a truly coupled design, it cannot do without losing growth, and that is the whole point.

Three things people get wrong

- ‘The model says the strain can make the product.’ Can is the upper edge of the envelope. Coupling is about the lower edge — what it must make.
- ‘More knockouts, more product.’ More coupling, less growth: 8.23 at μ 0.134 against 9.36 at 0.048.
- ‘Transport is a detail for later.’ Cap the export at a realistic rate and a coupled strain can become infeasible outright.

Self-check — answer on paper first

1. Explain the bilevel structure and why the inner problem is the cell's.

The outer problem is the engineer's: choose a set of deletions, within a budget, to maximise product. The inner problem is the cell's: given those deletions, distribute flux to maximise growth. The nesting matters because the engineer does not control the flux distribution — the cell does, and it will do whatever is best for itself. A design evaluated by the product the cell *could* make is wishful; a design evaluated by the product it must make while maximising growth is a prediction about a strain that will stay that way under selection.

2. Why did the aerobic search find no coupled design?

Because with oxygen freely available the cell can oxidise substrate completely to CO_2, generating ATP through respiration and balancing redox without any fermentative product, so there is always a growth-maximising solution with zero product. Coupling requires that every route for balancing carbon, redox and energy pass through the product, which under aerobic conditions would need the respiratory chain itself restricted — an oxygen limit, or deletions in respiration, which carry their own costs. Hence anaerobic or micro-aerobic designs for fermentative products.

3. A coupled design is predicted at 9 mmol gDW^{-1} h^{-1} of product. Your transporter can manage 5. What happens?

By the table above, somewhere between 4 and 6 the strain stops growing: the deletions make product formation obligatory, so if the product cannot leave, its intracellular pool cannot balance and there is no feasible steady state. Practically the strain will either not grow, accumulate the product internally to a toxic concentration, or acquire suppressor mutations that break the coupling — all three of which look like "the design did not work". The fix is to raise export capacity first, by overexpressing or screening a transporter, and only then to install the knockouts.

What to open a book for. Burgard, Pharkya & Maranas, Biotechnol. Bioeng. 84:647 (2003) for OptKnock, read for the bilevel formulation rather than the solver. A ^{13}C-MFA methods review for chapter 43's material, and the GECKO methodology paper for enzyme constraints. Part 10's chapters 35 and 36 are the machinery this chapter uses.

Retrosynthesis and pathway prediction: expanding a network backwards

A retrosynthesis tool proposes routes. What it cannot propose is an enzyme that exists, is active on your substrate, and folds in your host — which is where every one of its routes actually fails.

Reaction rules, and expanding backwards

RetroPath2.0 and its relatives work from **reaction rules**: generalised patterns abstracted from known enzymatic reactions, encoding which bonds change and in what chemical context, at a chosen level of specificity. Applied in reverse to a target molecule, a rule proposes precursors; applied again to those, it proposes their precursors, and so on, expanding a network backwards until it reaches compounds the host already makes.

The arithmetic of that expansion is why ranking matters more than generation. If each compound matches on average r applicable rules and each application yields one precursor set, a breadth-first expansion grows as r per generation: at r = 5, four generations is 625 leaf compounds and several thousand candidate routes; at r = 20, 160,000. Tools control this with rule-specificity settings, with a maximum number of steps, and by pruning against a sink of compounds known to exist in the host. The output is therefore not "the route" but a ranked list whose length is set by the settings you chose.

Ranking criteria, and what each is worth:

Criterion	What it measures	Limit
Thermodynamic feasibility	ΔG′ per step and the max-min driving force of the route (part 2 chapter 14)	Group-contribution estimates carry real uncertainty, especially for unusual chemistry
Enzyme availability	Whether a characterised enzyme catalyses each step, and on how similar a substrate	A rule that fired does not mean an enzyme exists; this is the binding constraint
Route length	Number of heterologous steps	A proxy for build effort and for the chance that one step fails
Host fit	Whether the precursor is abundant in the chosen host and the cofactors match	Needs the part 5 constraint sheets: which of the twelve precursors, and what cofactor demand
Toxicity of intermediates	Predicted or known toxicity	Usually unknown for novel intermediates

I could not run RetroPath2.0 or any retrosynthesis tool here — it needs network access and a rule database — so this chapter gives the reasoning and the arithmetic rather than output. The exercise the tracker sets is in any case a manual one, and it is the more valuable half.

Proposing a route by hand, and where the risk is

Take a non-natural target and propose two routes from a central metabolite. The method: identify the target's functional groups and carbon skeleton; find the nearest central-metabolism intermediate with the same skeleton or one bond away (the twelve precursors of part 1 chapter 7 are the candidate list); then work backwards one chemical transformation at a time — a reduction, a transamination, a decarboxylation, a condensation — writing down what enzyme class each would need.

Then source a candidate enzyme for every step from BRENDA or MetaCyc, and **mark every step for which you cannot find one**. Those marks are the project's real risk, and the arithmetic is unforgiving: if each step with a plausible enzyme has an independent probability p of actually working in your host, a route of n such steps works with probability p^n. At p = 0.8, a five-step route is 33%; at p = 0.6, 8%. A step with *no* candidate enzyme is not a low probability, it is a research project of its own — an enzyme discovery or design problem, chapter 29's territory — and a route containing two of them is a thesis rather than a pathway.

That is why route length and enzyme availability dominate the ranking in practice, and why a thermodynamically elegant six-step route with two unknown enzymes loses to a clumsy three-step route with three characterised ones.

Where machine learning actually enters the loop

The honest map, which chapters 41 and 42 then make concrete:

- **Design**: ranking retrosynthesis routes; predicting enzyme–substrate compatibility; predicting which variants of an enzyme are worth building (the variant-effect scores of part 8 chapter 29, which enrich a library roughly thirteen-fold

at a correlation of 0.5).

- **Build**: almost nothing. This is chemistry and robotics.
- **Test**: image and spectrum analysis, peak calling, outlier detection in plate data.
- **Learn**: the surrogate model that maps a design to a measured outcome and recommends the next batch — chapter 41 — which is where the phrase "machine-learning-guided DBTL" actually lives.

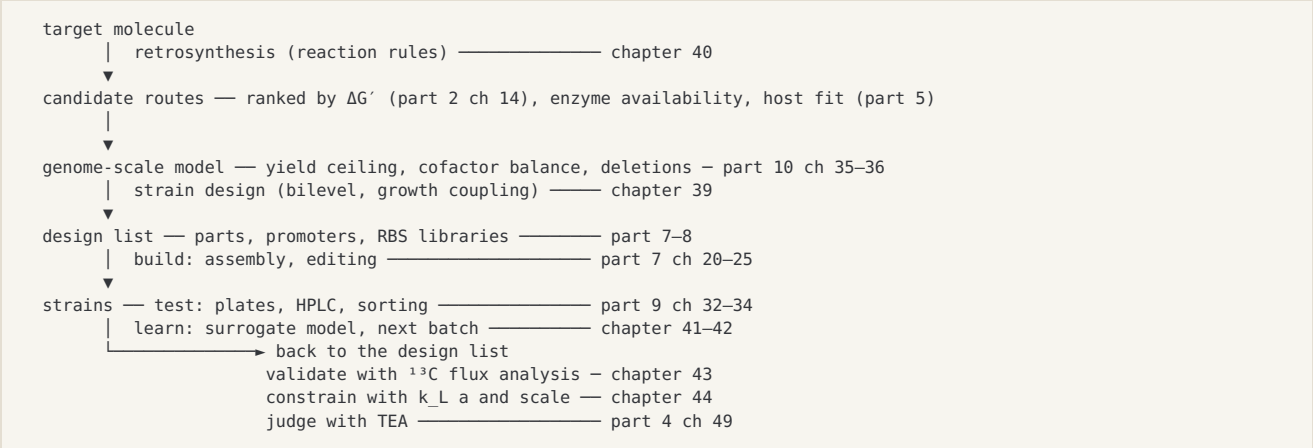
What it needs is the part most reviews skip: **data with identifiers that survive the pipeline** (part 9 chapter 34), enough of them, and a readout that tracks what you care about. Tens of experiments is the normal scale, which is why the methods that work here are the small-data ones of chapter 41 rather than the deep networks of the courses.

The tracker asks you to mark every forward-looking claim in a review as demonstrated or projected, as in part 6 chapter 61. In a 2025 review of AI-guided biofoundries expect the demonstrated set to be narrow — automated build and test at scale in specific well-instrumented facilities, surrogate models recommending batches in published campaigns — and the projected set to be most of the rest, especially anything described as autonomous, self-driving or closed-loop across the whole cycle.

What a biofoundry round actually looks like

Part 9 chapter 34 did the pipeline arithmetic; here the point is the division of labour. Automated: DNA assembly set-up, transformation plating in some facilities, colony picking, inoculation, plate-reader and sometimes chromatographic assay, data capture. Not automated: deciding what to build, designing the parts, troubleshooting a failed assembly, judging whether a hit is real, and the fermentation characterisation of a promising strain. For a PhD student's daily work this means the time saved is in repetitive execution, and the time spent moves to design and interpretation — which is exactly where the value of understanding parts 2 to 10 lies.

The computational stack, in one diagram



Expansion	r applicable rules per compound → r^n leaves after n generations
Route risk	P(route works) ≈ p^n; 0.8^5 = 33%, 0.6^5 = 8%
Unknown enzyme	not a low probability — a separate discovery project
Ranking that matters	enzyme availability and length first, thermodynamics second, elegance never
ML in the loop	design (ranking, variant priors) and learn (surrogate, next batch); not build

A tool returns 400 routes to your target. How do you get to three worth building?

Filter before you rank. First, **length**: discard anything longer than about five heterologous steps, which removes most of the list and costs little, since long routes fail on the p^n arithmetic anyway. Second, **precursor**: keep only routes starting from a precursor your host supplies well — part 5's constraint sheets say which, and a route from erythrose-4-phosphate in a host that has little of it is a project about precursor supply, not about the route. Third, **enzyme evidence**: for each remaining route, look up every step in BRENDA or MetaCyc and count the steps with a characterised enzyme on a substrate within one functional group of yours. Routes with any step at zero go to the bottom.

Then rank what survives, perhaps a dozen routes, by max-min driving force (part 2 chapter 14) and by cofactor demand against the host's supply (part 2 chapter 19) — and pick three that differ from each other, rather than three variations of the same chemistry, so that a shared failure does not kill all three. Finally, write down for each the single step you most expect to fail and how you would know: that sentence is worth more than the ranking.

Three things people get wrong

- ‘The tool found a route.’ It found a sequence of rule applications. Whether an enzyme exists for each is a separate question, and it is the one that decides the project.
- ‘Shorter is better because it is elegant.’ Shorter is better because the probability of the whole route working is roughly p to the power of its length.
- ‘Machine learning designs the pathway.’ It ranks candidates and recommends the next batch; the constraints that kill routes are thermodynamic, enzymatic and host-specific.

Self-check — answer on paper first

1. Why does ranking matter more than generation in retrosynthesis?

Because generation is combinatorial and cheap: a few rules applied over a few generations produce hundreds to hundreds of thousands of candidate routes, nearly all of which are unbuildable. The scarce resource is the attention to evaluate them and the years to build one, so the value of a tool lies in how well it orders its output by the things that actually kill routes — missing enzymes, thermodynamically stuck steps, precursors the host lacks and toxic intermediates — not in how many routes it can enumerate.

2. Two routes: four steps with all enzymes characterised, or two steps with one unknown enzyme. Which do you build?

The four-step route, in almost every case. At a generous $p = 0.8$ per characterised step it has a 41% chance of working as a whole, and every step has a literature starting point and a measurable activity. The two-step route's unknown enzyme is not a 50% risk, it is an open-ended discovery or design problem — screening homologues, directed evolution or computational design (part 8 chapters 29 and 30) — which can consume the whole project. The two-step route becomes attractive only if the unknown step is a well-precedented reaction class where a homologue is likely to be found quickly.

3. Where in the DBTL cycle does machine learning have the least to offer, and why?

Build. Assembly, editing and transformation are deterministic chemistry and robotics whose failure modes are physical — a bad overhang set, an inhibitor in the DNA, poor competence — and are diagnosed by the controls of part 7 chapter 20 and part 9 chapter 32, not by a model. The leverage is in design, where a prior narrows a library, and in learn, where a surrogate chooses the next batch from expensive measurements.

What to open a book for. Delépine et al., Metabolic Engineering 45:158 (2018) for RetroPath2.0 and how reaction rules expand a network; a 2025 review on machine learning or AI-powered biofoundries, read with a pen for demonstrated against projected; and BRENDA and MetaCyc, which are where the enzyme-availability question is actually answered.

Bayesian optimisation: choosing the next experiment from a dozen noisy points

The constraint in strain engineering is not data volume, it is experiment count. Twenty runs, each a week of work and a few per cent noisy, is the problem — and it is the opposite of the machine learning most courses teach.

The problem this is actually solving

The Automated Recommendation Tool paper's framing is the one to absorb: a handful of expensive experiments, each producing one noisy number, with a design space too large to grid and interactions between factors that make one-at-a-time optimisation wrong (part 2 chapter 17). That is not a big-data problem. It is a **sequential decision problem under uncertainty**, and the method that fits it is a surrogate model with calibrated uncertainty plus a rule for choosing where to look next.

Count the experiments in your own plan. If the answer is a few dozen, every method that needs thousands of examples is irrelevant, and the question becomes how to spend the few you have.

Gaussian process surrogates

A Gaussian process is a distribution over functions: given observations, it returns at every untried point a posterior **mean** — the best guess — and a posterior **standard deviation** — how unsure it is. The kernel encodes the assumption about smoothness: a Matérn kernel with a length scale of 0.25 on a unit-scaled factor says that the response changes appreciably over about a quarter of the factor's range and is continuous but not infinitely smooth. Far from any data the posterior mean reverts to the prior and the standard deviation grows to the prior's — which is the property that makes sequential design possible at all, because a model that reports "I do not know, here" can be asked to go and find out.

```
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import Matern, ConstantKernel, WhiteKernel
k = ConstantKernel(1.0) * Matern(length_scale=0.25, nu=2.5) + WhiteKernel(noise**2)
gp = GaussianProcessRegressor(kernel=k, normalize_y=True, n_restarts_optimizer=3).fit(X, y)
mu, sd = gp.predict(Xgrid, return_std=True)
```

The WhiteKernel term matters: it tells the model the measurements are noisy, so it does not interpolate every point exactly. Without it, a noisy replicate becomes a spike the model believes.

Choosing the next experiment

An acquisition function turns the posterior into a decision. **Expected improvement** is the expected amount by which a point would beat the best result so far, integrating over the posterior:

```
EI(x) = (μ(x) − y_best − ξ)·Φ(z) + σ(x)·φ(z),      z = (μ(x) − y_best − ξ)/σ(x)
```

The first term rewards a high predicted mean — exploitation — and the second rewards high uncertainty — exploration; ξ is a small constant that biases toward exploration. **Upper confidence bound**, μ + κσ, does the same more bluntly with κ setting the balance. Either way, the next experiment is the arg max, and the loop is: fit, maximise the acquisition, run that experiment, repeat.

The comparison, run forty times

I built a two-factor medium optimisation: a response surface with a narrow global optimum (titer 3.03) and a broader local one (2.1), with measurement noise of 0.12 — about 4% at the optimum. Then I ran Bayesian optimisation from five random initial points against a 16-run factorial grid on the same experimental budget, forty seeds each.

Method and budget	Median best titer	10–90% range	Reaching ≥95% of the best
Bayesian, 8 runs	2.32	1.93–2.99	22%
Bayesian, 11 runs	2.98	2.08–3.02	62%
Bayesian, 16 runs	3.01	2.09–3.03	68%
16-run factorial grid	2.35	2.35–2.35	0%

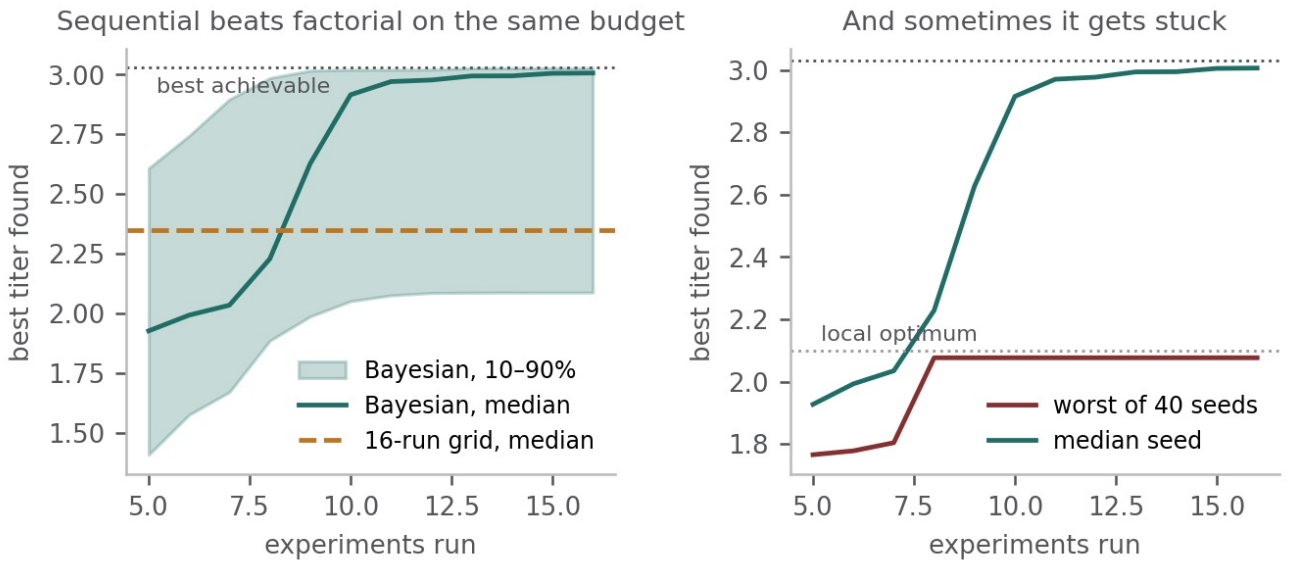


Figure 41.1 Left: best titer found against experiments run, median and 10–90% band over 40 seeds, against the 16-run grid. Right: the median seed against the worst of the 40, which converged on the local optimum and stayed there. Computed for this module; the response surface, the noise level and the kernel are stated in the text.

Three readings. **The sequential method wins decisively in the median** — by eleven runs it is at 98% of the achievable best, which the grid never reaches. **The grid's spread is zero** because it always visits the same points: it is reproducible and reproducibly wrong, missing the narrow optimum because no grid node lands near it. And **the spread of the sequential method is large**: the 10th percentile at sixteen runs is 2.09, barely above the local optimum. Reporting your best seed would be dishonest in exactly the way part 2 chapter 17 warns about.

Where it fails: the stuck seed

In the worst of forty seeds the run ends at 2.09 — 69% of the achievable best — having converged on the local optimum. What the acquisition function did is the general failure: once several points cluster around a good region, the posterior standard deviation there collapses, the mean is high, and expected improvement keeps selecting nearby points because each has a small but positive chance of beating the current best. Meanwhile the unexplored region retains high uncertainty but a *low mean*, and with ξ small, exploitation wins. The defences are a larger ξ, an upper-confidence-bound rule with a generous κ, batch selection that deliberately includes a far-away point, and more initial design points before the loop starts.

When not to use it

- **Fewer than about a dozen runs in total.** With five initial points and a handful of iterations the surrogate is barely constrained; a careful factorial or a one-factor sweep is more defensible and easier to explain.
- **A noisy or drifting assay.** The method assumes the noise model it was given. A drift — part 9 chapter 34's plate position effects, or chapter 50's instrument drift — is read as signal, and the optimiser will chase it.
- **Confounded factors.** If two factors always move together, the surrogate cannot separate them and will recommend along the confounded direction.
- **A readout that does not track what you care about.** The optimiser maximises what you measure, which is part 8 chapter 28's rule in a new form: a biosensor proxy optimised for twenty runs returns a strain that is excellent at the proxy.

From factors to sequences

Medium components are continuous and low-dimensional; a promoter or ribosome binding site choice is categorical and high-dimensional. To let a model learn across variants you must **encode the genotype**, and the options run from crude to subtle: one-hot indicators per part, which treats every part as unrelated; measured or predicted strengths as continuous features, which lets the model interpolate between a weak and a strong promoter and is usually the best value for the effort; physicochemical or sequence descriptors for an RBS; and embeddings from a pretrained sequence model for a protein variant (part 8 chapter 29). The tryptophan work the tracker names is the hybrid case worth reading: a mechanistic model of the pathway supplies structure, and the learned component covers what the mechanism does not.

Two sentences for a capstone build-and-test plan, which most proposals leave as the word "learn": *After each round of twelve strains we will fit a Gaussian process to titer as a function of the three promoter strengths, encoded as measured relative promoter units, and select the next twelve by expected improvement with a deliberately large exploration constant, keeping two of the twelve as replicates of the current best for drift control. We will report the median and spread across rounds, and stop when the predicted improvement falls below the assay's reproducibility.*

Surrogate	`Gaussian process: posterior mean and standard deviation at every untried point`
Noise	`a WhiteKernel term, or the model interpolates noise`
Expected improvement	`(μ − y_best − ξ)Φ(z) + σφ(z): exploitation plus exploration`
Result here	`11 runs reached 98% of the best; a 16-run grid reached 78% and never more`
Failure	`1 seed in 40 stuck at the local optimum, 69% of the best`
Report	`the spread over seeds, not your best run`

A colleague has run eleven medium experiments by Bayesian optimisation and wants to stop, claiming the optimum is found. What do you ask?

Three questions. **What does the surrogate predict for the best untried point, and with what uncertainty?** If the predicted improvement is smaller than the assay's reproducibility, stopping is defensible; if the model still sees a region with a high upper bound, it is not. **Where did the eleven points go?** If nine of them cluster in one region, the result may be a local optimum like my worst seed — the test is cheap: run two deliberately distant points and see whether the model was right that they are poor. **What is the replicate noise?** Without it there is no way to know whether the best point is genuinely best or a lucky draw, and part 2 chapter 17's winner's-curse argument says the best of eleven noisy measurements is biased upward.

Then the honest framing for a report: not "the optimum is 3.0" but "the best measured condition gave 3.0 ± noise, the surrogate predicts no untried point above that by more than the assay can resolve, and the search covered this region of the factor space" — with the coverage shown, because a reader cannot otherwise tell a thorough search from a lucky one.

Three things people get wrong

- ‘We used machine learning, so we need a lot of data.’ This method is for tens of runs. The kind that needs thousands is a different problem.
- ‘The grid is safer because it is systematic.’ It is reproducible and it missed the optimum in 40 of 40 seeds, because no node landed near it.
- ‘It converged.’ One seed in forty converged on the wrong peak. Report the spread, and test the model's claim that the unexplored region is poor.

Self-check — answer on paper first

1. Why does a surrogate need to report uncertainty for sequential design to work at all?

Because the decision is where to spend the next experiment, and that requires knowing not only where the response is predicted to be high but where the prediction could be wrong. A model that returns only a mean cannot distinguish a point it knows to be mediocre from a point it has no information about, so it has no basis for exploring. The posterior standard deviation is what turns "I predict 2.0 here" into "I predict 2.0 ± 0.1 here and 1.8 ± 0.9 there", and the second point may well be worth the run.

2. Your acquisition function keeps proposing points next to the current best. Diagnose and fix.

The exploitation term is dominating: the cluster of points has collapsed the local uncertainty, the mean there is the highest known, and expected improvement with a small ξ will keep picking neighbours because each has a positive chance of a marginal gain. Fix by raising the exploration constant ξ, switching to upper confidence bound with a generous κ, selecting a batch that is required to include a point far from the existing data, or simply adding a few space-filling points by hand. The underlying question to ask is whether the response surface plausibly has a second peak — if it does, as in my simulation, the cost of not exploring is 31% of the achievable titer.

3. When would you choose a factorial design over sequential optimisation?

When you need to *understand* rather than optimise: a factorial estimates main effects and interactions, which is what you report when the question is which factors matter and how they interact. Also when the total run count is very small, where a surrogate is barely constrained; when factors are categorical with few levels, where there is nothing to interpolate; and when the runs must be executed in one batch for logistical reasons, since sequential methods need rounds. The two combine well: a small factorial to find which factors matter, then sequential optimisation over those.

What to open a book for. Radivojević et al., Nat. Commun. (2020) for the Automated Recommendation Tool and its framing; Zhang et al., Nat. Commun. (2020) on tryptophan for the mechanistic-plus-learned hybrid; and the scikit-learn Gaussian process documentation, which is the practical reference for the kernels and the `return_std` argument that makes all of this work.

Chapter 42 · tracker week 42

Machine learning for biology: overfitting, leakage and the split that matches the question

Two simulations settle most of this week. One shows a model with a perfect training score and no predictive power. The other shows the same data giving R^2 of 0.67 or −0.44 depending only on how the test set was chosen.

A model, a feature, a baseline

A model is a function fitted to examples. What varies in biology is the **features** — the numbers that stand for the thing. For a **sequence**: one-hot encoding per position, physicochemical descriptors per residue, k-mer counts, or an embedding from a pretrained protein language model. For a **strain**: which promoter and ribosome binding site in which position, as measured strengths rather than labels (chapter 41), plus the deletions as indicators. For a **plate**: well position, row, column, edge indicator, plate identifier and run date — which are features precisely because they predict the outcome, and that is the problem this chapter is about.

The baseline comes first. Fit the mean, then a linear or ridge model, then nearest neighbours, and report the held-out error of each. A complex model has to beat them on held-out data to be worth anything, and in small biological datasets it frequently does not. This is not modesty; it is the only way to know whether the fancy model learned anything, because an R^2 of 0.6 is impressive against a mean-only baseline of 0 and unimpressive against a ridge baseline of 0.58.

How much data is enough: the overfitting simulation

Sixty features, of which five carry real signal, with noise, fitted by ridge regression and evaluated on 400 held-out samples, averaged over thirty repeats:

Training samples	Training R^2	Held-out R^2
20	1.00	0.19
40	1.00	0.40
80	0.97	0.64
160	0.93	0.82
320	0.91	0.87

At twenty samples and sixty features the model fits its training data perfectly and predicts almost nothing: **a training R^2 of 1.00 with a held-out R^2 of 0.19**. The gap closes as samples accumulate, and the learning curve flattens around 160 to 320 — which for this noise level and this number of features is where more data stops paying.

Then the control that makes the point unarguable: **shuffle the labels** and refit. With n = 40 the training R^2 is still 1.00 and the held-out R^2 is −1.67 — worse than predicting the mean. A model can fit pure noise perfectly whenever it has enough parameters, so a high training score is not weak evidence of learning, it is *no* evidence. Run the shuffle on your own data as a matter of routine: if the shuffled model's held-out performance is not near zero, your validation is broken.

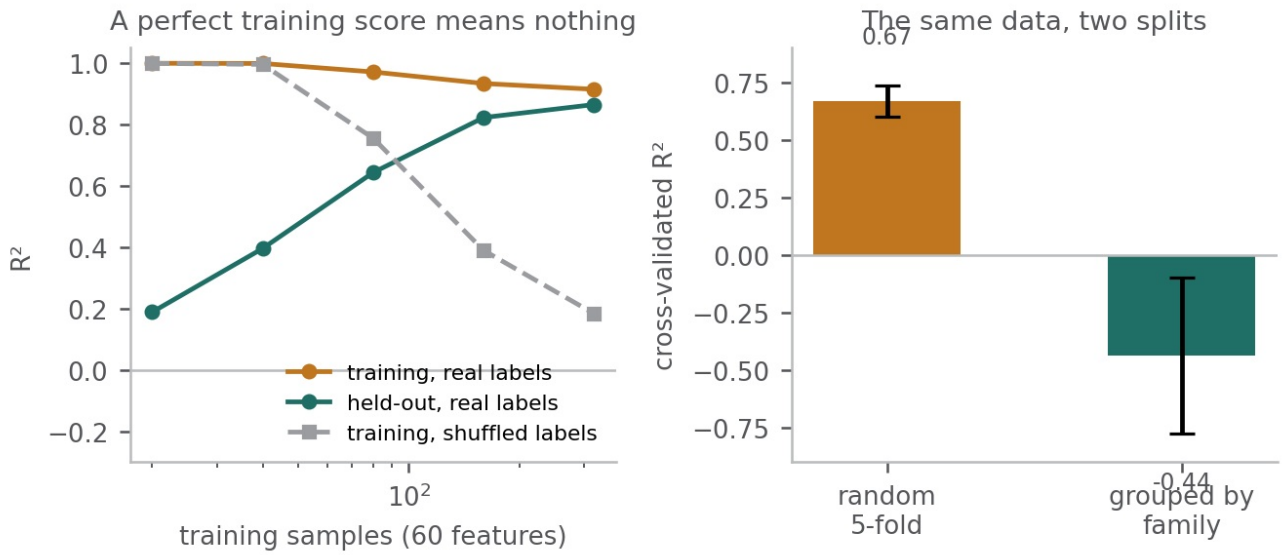


Figure 42.1 Left: training and held-out R² against training-set size for ridge regression with 60 features and 5 carrying signal, with the shuffled-label control. Right: the same dataset of 40 families × 6 variants evaluated by random five-fold cross-validation and by grouping folds by family. Computed for this module; parameters in the text.

Leakage: the same data, two answers

Now the simulation that matters most for sequence work. Forty families, six variants each, where the family identity accounts for most of the variance and the within-family differences are small — which is exactly the structure of a protein dataset built from homologues, or a strain dataset built from a few parental backgrounds. A random forest, evaluated two ways:

Cross-validation scheme	R²	What it means
Random 5-fold	**0.67 ± 0.07**	Relatives of every test point are in the training set; the model recognises the family and reports its mean
Grouped by family	**−0.44 ± 0.34**	Whole families held out; the model has learned nothing that generalises to a new family

The same data, the same model, and a swing from "works well" to "worse than the mean". The random split leaks because the information that predicts a test point is not the biology but the presence of its siblings in the training set. A negative R² under the grouped split is the honest answer to the question "will this predict a family I have not seen", and it is *no*.

Which split matches which question, which is the week's real content:

The question	The split
Rank variants within families I already have data for	Random split is legitimate — the use case matches the evaluation
Predict a family I have never measured	Group by family, cluster or sequence identity threshold
Predict next week's plate, or a new batch	Split by plate, by run date, or forward in time

One way my own data could leak: strains built from the same parental background and measured on the same plate share both a genetic and a positional effect, so a random split over wells reports a model that has learned the plate. What to report so a reader can tell: the split rule, in one sentence in the methods — *folds were grouped by parental strain so that no two variants of a background appear in both training and test, and plate identity was additionally held out* — plus both numbers when they differ, which is more informative than the better one alone.

Pretrained models and active learning

A **zero-shot score** is a prediction made with no training on your protein at all: a protein language model assigns each substitution a likelihood from patterns learned across evolution, and that correlates partially with measured fitness (part 8 chapter 29, where a correlation of 0.5 enriched the true top 1% thirteen-fold and cut a 30,000-clone screen to about 2,300). What it cannot tell you is anything about *your enzyme in your host*: expression, solubility, the non-natural substrate you care about, or the cofactor you swapped — because none of that is in the evolutionary record it learned from.

Active learning and Bayesian optimisation are the same idea with different emphasis. Chapter 41's acquisition function chooses the next point to **maximise an outcome**; active learning chooses the next batch to **reduce the model's**

uncertainty, which is what you want when the goal is a predictive model rather than one good strain. Both need the same thing: calibrated uncertainty.

When a model prior and a structure prior agree or conflict: they agree when a substitution is both evolutionarily unusual and buried in the core — strong evidence against it. They conflict when a substitution is evolutionarily rare but structurally innocuous, which is common for positions under selection for something other than stability, such as a regulatory interaction you do not care about. The resolution is that the structure prior answers "will it fold" and the sequence prior answers "has nature tolerated it", and for an engineering goal — a new substrate, a swapped cofactor — nature's answer may be irrelevant because nature was not trying to do what you are.

A notebook that stays honest

The one page the week asks for, with the seven headings filled: **the question** — stated as a prediction task, including what a new data point looks like; **the data** — how many, from where, with what replication; **the split** — the rule and why it matches the question; **the baseline** — mean, linear, nearest neighbours, with their held-out scores; **the metric** — and why, since R^2 on a skewed target flatters a model that predicts the bulk; **the uncertainty** — the spread across folds and seeds, not the best; and **what would change your mind** — stated in advance, which is the hardest and most valuable line.

Add seeds and pinned versions, and check the notebook runs from a fresh clone, as part 10 chapter 36 required. And the methods sentence about the split, written once and reused.

On AI assistants, which the tracker raises explicitly: they write plausible code and plausible prose, and the failure mode here is specific — code that touches the split or the metric can be subtly wrong in a way that inflates the score, and a wrong citation is invisible until a reviewer checks. The checks worth doing, which I applied to my own code in this part: run it on a case with a known answer (the shuffled-label control is exactly that), read every line that touches the split, the leakage structure or the metric, and confirm that any citation exists. Write down what the assistant did and what you checked, because that sentence is what makes the work yours.

Overfitting	`n = 20, p = 60: training R² 1.00, held-out 0.19`
Shuffle control	`shuffled labels still fit perfectly in training; held-out R² −1.67`
Leakage	`random 5-fold 0.67 against grouped-by-family −0.44, same data`
Split rule	`match the split to what you must predict: variant, family, or plate`
Zero-shot score	`a prior worth an order of magnitude in screening; silent about your host`
Baseline	`mean, linear, nearest neighbours — reported, before anything complex`

A collaborator reports $R^2 = 0.8$ predicting enzyme activity from sequence, on 300 variants from twelve homologue families, with random five-fold cross-validation. What do you conclude?

That the number is almost certainly a family-recognition score rather than a prediction of activity. With twelve families and 300 variants there are twenty-five variants per family, so a random split puts relatives of every test point in the training set, and my simulation shows exactly this structure giving 0.67 randomly and −0.44 when grouped. The first thing to ask for is the grouped-by-family number. If it is near zero, the model predicts the family mean and nothing else, which is still useful for ranking within a known family and useless for a new one.

Then two further checks: the shuffled-label control, which should give a held-out score near zero and will reveal any residual leakage in the pipeline; and a baseline, because if a ridge model on family identity alone reaches 0.78, the sequence features added two points. And the question behind all of it — what will this model be used for? If the plan is to rank variants of a measured enzyme, the random split is the right evaluation and the number stands. If the plan is to pick a new homologue to engineer, it does not.

Three things people get wrong

- ‘The training score is excellent.’ It is also excellent on shuffled labels. Only held-out performance is evidence.
- ‘We cross-validated.’ With what split? Random folds over related sequences measure family recognition, as 0.67 against −0.44 shows.
- ‘The language model scored this variant highly.’ It scored it against evolution, which was not trying to make your product in your host.

Self-check — answer on paper first

1. Why does shuffling the labels diagnose a broken validation?

Because with shuffled labels there is, by construction, no relationship between features and target, so any correctly validated model must score near zero on held-out data. If it does not — if the shuffled model still appears to predict — then information is reaching the test set that should not be: leakage through related samples, a preprocessing step fitted on the whole dataset before splitting, or a feature that encodes the target. The training score, meanwhile, can be perfect on shuffled labels whenever the model has enough capacity, which is why it is not evidence of anything.

2. You must predict titer for strains built from a new parental background. Design the evaluation.

Group the folds by parental background, so that every test fold contains backgrounds entirely absent from training — that matches the deployment. Report the per-fold scores and their spread, not the pooled number, because backgrounds differ and a single average hides a fold where the model failed completely. Include a baseline that predicts the global mean and one that predicts from the background's own mean where available, to show what the sequence or design features add. And if plates differ between backgrounds, hold plate out as well, or the background effect and the plate effect are confounded.

3. When is a random split the right choice?

When the intended use really is to interpolate within the population you sampled: ranking further variants of the same enzyme families you have measured, or predicting an untested combination of promoter strengths within a library whose parts are all represented in training. The split should mirror the deployment, and if the deployment involves known families then relatives in the training set are a feature rather than leakage. The error is not using a random split; it is using one and then claiming the model generalises to something it was never tested on.

What to open a book for. The literature searches the tracker names — machine-learning-guided directed evolution, protein language models and fitness prediction, and data leakage with homology-aware splitting — are the sources, and the third is the one to read first. Part 4 chapter 17 and part 8 chapter 29 of this module are the chapters this one builds on.

^{13}C metabolic flux analysis: what labelling resolves that stoichiometry cannot

Stoichiometry says what is possible, measured uptake rates pin the boundary, and labelling tells you what actually happened inside. The third is the only one that can falsify a flux map.

Three kinds of flux, and what only labelling settles

Part 10 chapter 35 found fourteen reactions in a 40-reaction model with non-zero flux variability at exactly optimal growth: the network and the objective did not determine them. Measured exchange rates — the uptake of glucose and oxygen, the secretion of by-products — pin the boundary fluxes and leave much of the interior free. Labelling resolves the interior, and three things in central metabolism depend on it entirely.

- **The split at glucose-6-phosphate** between glycolysis and the oxidative pentose phosphate pathway, which decides NADPH supply and carries the carbon cost of part 1 chapter 5.
- **Reversible exchange fluxes**: a near-equilibrium reaction's net flux is a small difference between two large opposing fluxes (part 2 chapter 14), and only labelling sees the exchange.
- **Anaplerotic against cyclic TCA flux**: carbon entering oxaloacetate by carboxylation against carbon arriving round the cycle, which stoichiometry cannot distinguish because both produce the same metabolite.

Tracer design

[1-^{13}C] glucose labels one specific carbon, and its value is positional: carbon 1 is released as CO_2 by 6-phosphogluconate dehydrogenase in the oxidative branch, and survives through glycolysis to become a methyl carbon of pyruvate. So the labelling of downstream metabolites reports the split directly, and it is the classic tracer for that question. **[U-^{13}C] glucose**, uniformly labelled, reports on how intact carbon skeletons are cleaved and recombined — bond breaking and forming — because a fragment's mass shift tells you how many of its carbons came from the same original molecule; it is what resolves anaplerosis and the cyclic flux. Mixtures — commonly a defined ratio of [1-^{13}C] and [U-^{13}C] with unlabelled glucose — are used because no single tracer resolves everything, and the mixture is chosen to maximise the information for the specific fluxes in question.

For the pentose phosphate split, the justification in two sentences: *[1-^{13}C] glucose places the label on the carbon that the oxidative branch decarboxylates, so any flux through that branch removes label from the pool while glycolysis retains it, and the fractional labelling of pyruvate therefore varies monotonically with the split. A mixture with [U-^{13}C] would be added if anaplerotic flux also had to be resolved in the same experiment.*

Atom transitions, and the carbon that is lost

The exercise the tracker sets is to write the transitions carbon by carbon. Glucose carbons 1 and 6 become the methyl carbons of the two pyruvates; carbons 2 and 5 the carbonyl carbons; carbons 3 and 4 the carboxyl carbons, which are the ones released as CO_2 by pyruvate dehydrogenase. In the oxidative pentose phosphate branch, glucose-6-phosphate is oxidised at carbon 1 and then **carbon 1 is released as CO_2** by 6-phosphogluconate dehydrogenase, leaving a five-carbon ribulose-5-phosphate whose carbons derive from glucose 2 to 6. That asymmetry is the entire basis of the measurement.

A **mass isotopomer distribution** is the set of fractions M+0, M+1, M+2 … of a measured fragment containing zero, one, two or more heavy atoms. It is what a mass spectrometer reports, and it must be **corrected for natural abundance** before it means anything: carbon is 1.07% ^{13}C naturally, so a three-carbon fragment has an M+1 of about 3.2% before any tracer is added, and larger fragments more. Silicon and sulfur in derivatisation reagents contribute further. Skipping the correction biases every flux.

Fitting a split, with its confidence interval

I built the simplest possible version to make the statistics concrete: a model in which a fraction f of glucose-6-phosphate goes through the oxidative branch, with [1-^{13}C] glucose, predicting the M+1 fraction of pyruvate including the natural-abundance contribution.

Oxidative PP fraction	Predicted pyruvate M+1
0.00	0.511

0.10	0.463
0.30	0.367
0.50	0.271
0.80	0.128

Fitting f from a measurement generated at a true value of 0.30 with 0.4% absolute noise on the MID gives **f = 0.298 ± 0.003**, with a bootstrap 95% confidence interval of **0.289 to 0.312**. The sensitivity is excellent because the response is steep: a 1% change in the split moves M+1 by about 0.005, which is larger than the measurement noise.

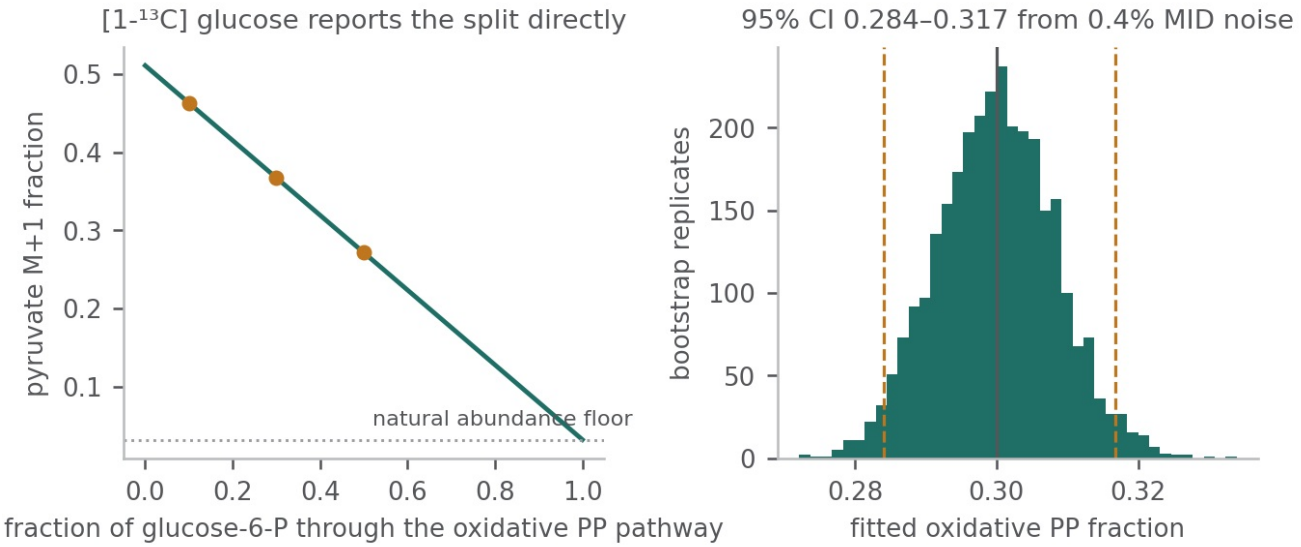


Figure 43.1 Left: predicted pyruvate M+1 fraction against the oxidative pentose phosphate fraction for [1-¹³C] glucose, with the natural-abundance floor marked. Right: the bootstrap distribution of the fitted split from 3,000 synthetic measurements at 0.4% MID noise. Computed for this module with a deliberately minimal atom-transition model — a real analysis uses the full network and software such as INCA, OpenFlux or mfapy, which I could not run here.

What a real fit does is the same thing on a network: simulate the labelling of every measured fragment from a candidate flux vector through the full atom-transition network, compare with the measured MIDs, and minimise the weighted sum of squared residuals over the free fluxes — **nonlinear least squares**, because labelling depends nonlinearly on fluxes. Confidence intervals come from the curvature at the optimum or, better, from a parameter scan: fix each flux at a grid of values, re-optimise the rest, and find where the residual rises past a chi-square threshold.

Reading the statistics honestly

A **poor goodness of fit** means one of two things and they need opposite responses. Either the data are bad — a mis-corrected natural abundance, a contaminated fragment, an unreached isotopic steady state — or **the network is missing a reaction**, in which case no flux vector can reproduce the labelling and the fit is telling you something valuable. The way to distinguish them is to look at which fragments are badly fitted: scattered residuals suggest data, while a consistent failure on fragments downstream of one node suggests a missing route there.

And the caution that matters for reading papers: **a narrow confidence interval on a badly specified network is worth nothing.** The interval reports how well the data constrain the fluxes *within the assumed network*, so a model lacking a real reaction can fit its own structure beautifully and precisely. What establishes the network is the goodness of fit together with the chemistry; what the intervals establish is precision given that.

Two assumptions underlie everything: **isotopic steady state**, that the labelling has stopped changing, which takes a few residence times and is slow for large pools; and **metabolic steady state**, that the fluxes themselves are constant during the labelling. Neither holds in a fed-batch, in a shifting culture, or in a photoautotroph with a slow-turning Calvin cycle — which is why non-stationary ¹³C-MFA exists, fitting the time course of labelling and thereby needing far more data and computation.

What labelling adds	`the G6P split, exchange fluxes, anaplerotic against cyclic TCA`
[1-¹³C]	`positional: C1 lost as CO₂ in the oxidative branch, retained through glycolysis`
[U-¹³C]	`bond breaking and forming; resolves anaplerosis`
MID	`M+0, M+1 … fractions; natural abundance is 1.07% ¹³C — correct it`

Fitting	nonlinear least squares over the atom-transition network; CIs from a parameter scan
Poor fit	bad data or a missing reaction — look at which fragments fail

A paper reports fluxes with tight confidence intervals and a goodness of fit that fails its chi-square test. How do you read it?

The failed fit is the headline and the intervals are decoration. A chi-square failure means no flux vector in the assumed network reproduces the measured labelling within the stated measurement errors, so either the errors are understated, the data are biased — most commonly by natural-abundance correction or by an unreached isotopic steady state — or the network omits a reaction that is carrying flux. The tight intervals are conditional on that network and simply say that, if the network were right, the data would pin the fluxes; they cannot detect the structural error.

What I would want: the residuals by fragment, to see whether the misfit is scattered or concentrated downstream of one node; confirmation that the culture was at isotopic and metabolic steady state, which for a batch culture sampled during growth is a real question; and the authors' own account of what they tried adding to the network. A paper that reports a failed fit, says so, and discusses the candidate missing reactions is more trustworthy than one that reports a perfect fit with no diagnostics — and the tracker's observation is usually right, that the measured fluxes contradict something in the authors' model, which is the most interesting sentence in the paper.

Three things people get wrong

- ‘The flux map is measured.’ Fluxes are *inferred* by fitting labelling to an assumed network; the network is an assumption the fit can test but not replace.
- ‘M+1 means one labelled carbon.’ Before natural-abundance correction, a three-carbon fragment has an M+1 of about 3% with no tracer at all.
- ‘Tight intervals mean reliable fluxes.’ They mean precision within the assumed network. Check the goodness of fit first.

Self-check — answer on paper first

1. Show why carbon 1 of glucose reports on the glucose-6-phosphate split.

In the oxidative branch, glucose-6-phosphate is oxidised to 6-phosphogluconate and then decarboxylated by 6-phosphogluconate dehydrogenase, and the carbon released as CO_2 is carbon 1 — so label at that position is lost from the carbon skeleton entirely. Through glycolysis, carbon 1 is retained: the hexose is cleaved so that carbons 1, 2 and 3 become one triose and 4, 5 and 6 the other, and carbon 1 ends as a methyl carbon of pyruvate. Therefore the fraction of pyruvate carrying the label falls monotonically with the fraction of glucose-6-phosphate routed through the oxidative branch, which is the measurement.

2. Why must MIDs be corrected for natural abundance, and what happens if you do not?

Because 1.07% of all carbon is naturally ^{13}C, along with heavy isotopes of hydrogen, oxygen, silicon and sulfur in the analyte and any derivatisation group, so every fragment shows M+1, M+2 and so on before any tracer is added — about 3.2% M+1 for three carbons, and more for larger fragments. Without correction the apparent labelling is systematically too high, so the inferred flux through the labelled route is overestimated, and the bias is larger for larger fragments, which distorts the *relative* information between measurements and therefore the whole flux map rather than just scaling it.

3. Your fit is poor. Give the two explanations and the diagnostic that separates them.

Bad data or a missing reaction. Look at the residuals fragment by fragment: if the misfit is scattered across unrelated fragments with no pattern, suspect the data — natural-abundance correction, instrument calibration, co-eluting contaminants, or a culture that had not reached isotopic steady state. If the misfit is concentrated in fragments derived from metabolites downstream of a particular node, suspect the network at that node — a missing bypass, an unaccounted exchange flux or a reaction with the wrong atom mapping. Adding the candidate reaction and seeing whether the fit becomes acceptable is the test, with the caveat from chapter 37 that a reaction added to make a model fit is a hypothesis to verify, not a finding.

What to open a book for. An Antoniewicz review of ^{13}C-MFA, which is the clearest account of the method and its statistics; a ^{13}C-MFA experimental protocol paper for the sampling, quenching and derivatisation that decide whether the data are any good; and a review of non-stationary ^{13}C-MFA if your organism is slow or your culture is not at steady state. The software — INCA, OpenFlux, mfapy — is where the real fitting happens, and none of it ran here.

Chapter 44 · tracker week 44

Bioreactor engineering and scale-up: mixing, transfer, feeding and the cost sketch

This week is mostly translation, not new theory — the transport phenomena you already have, applied to a vessel with cells in it. The single most important number is that mixing time grows with scale even at constant power per volume.

Power and mixing

For a stirred tank, the impeller Reynolds number is $Re = \rho n d^2/\mu$, with n the rotational speed and d the impeller diameter, and above about 10^4 the flow is fully turbulent and the power number N_p is constant — about 5 for a Rushton turbine. Then

```
P = N_p · ρ · n³ · d⁵
```

Compute two vessels at comparable power per volume. A 1 L vessel with a 4 cm impeller at 500 rpm (8.33 s^{-1}) draws 0.3 W, which is **296 W/m³**, with a tip speed of 1.05 m/s. A 100 m³ vessel with a 1.8 m impeller at 42 rpm (0.70 s^{-1}) draws 32 kW, which is **324 W/m³** — comparable power per volume — with a tip speed of 3.96 m/s.

Now the scaling that matters. Under geometric similarity the volume goes as d^3, so holding P/V constant requires $n^3d^5/d^3 = n^3d^2$ constant, giving $\mathbf{n \propto d^{-2/3}}$. Tip speed nd then rises as $d^{1/3}$ and the mixing time, which scales roughly as 1/n for a given geometry, **grows as $\mathbf{d^{2/3}}$**:

Impeller diameter	Speed at constant P/V	Tip speed	Mixing time, relative
0.04 m (1 L)	8.33 s^{-1}	1.05 m/s	1.0
0.40 m (~1 m³)	1.80 s^{-1}	2.26 m/s	4.6
1.80 m (~100 m³)	0.66 s^{-1}	3.72 m/s	12.7

So a vessel that mixes in a second or two at bench scale takes tens of seconds at cubic-metre scale at the same power per volume, and the only ways out are more power — which costs money and shear — or accepting the gradients. **That ratio is the whole scale-up problem in one number**, and the cell-level consequence is the next section but one.

Gas–liquid transfer, and the number that stops the process

The oxygen transfer rate is $OTR = k_La\cdot(C^* - C_L)$: the volumetric transfer coefficient times the driving force between saturation and the bulk dissolved concentration. k_La depends on power per volume, superficial gas velocity, and the medium's coalescence behaviour — correlations of the form $k_La = C(P/V)^{\alpha}v_s^{\beta}$ with exponents around 0.4 to 0.7 and 0.2 to 0.5 are standard; C* depends on temperature, pressure and the gas composition and is about 0.21 mM for air at 37 °C.

The demand side: a culture at 50 gDW/L with a specific oxygen uptake of 5 mmol·$gDW^{-1}\cdot h^{-1}$ consumes **250 mmol·$L^{-1}\cdot h^{-1}$**. Holding C_L at 0.05 mM leaves a driving force of 0.16 mM, so the required k_La is 250/0.16 = **1,560 h^{-1}**, against the roughly 100 to 500 h^{-1} a large stirred tank achieves. The culture cannot be supplied with air at atmospheric pressure.

Cell density	Oxygen uptake rate	Required k_La
10 gDW/L	50 mmol·$L^{-1}\cdot h^{-1}$	310 h^{-1} — feasible
20 gDW/L	100	625 h^{-1} — at or beyond the limit
30 gDW/L	150	940 h^{-1}
50 gDW/L	250	1,560 h^{-1}

The options, in the order a process engineer would consider them: **oxygen enrichment**, which raises C* in proportion to the oxygen mole fraction and can give a factor of four; **head pressure**, which raises C* proportionally — two bar absolute doubles it; **more power and gas flow**, which raises k_La sublinearly and costs utilities and shear; and **lowering the demand**, by feeding more slowly so the specific uptake rate falls. The last is usually the answer, and it is why nearly every industrial fermentation is a fed-batch — part 1 chapter 6's conclusion arriving as a reactor constraint.

Feeding the constraint into the model

This is the week's best exercise and the point where the transport calculation and the genome-scale model meet. Convert an achievable OTR into a per-biomass flux and impose it as the oxygen exchange bound in the core model of part 10, at 10 gDW/L:

k_La	OTR	Oxygen available	Growth	Acetate	Ethanol
100 h^{-1}	16 mmol·L^{-1}·h^{-1}	1.6 mmol·gDW^{-1}·h^{-1}	0.313 h^{-1}	4.92	6.78
200	32	3.2	0.409	4.47	4.68
300	48	4.8	0.505	4.01	2.58
500	80	8.0	0.683	1.90	0
1,500	240	24.0	0.754	0	0

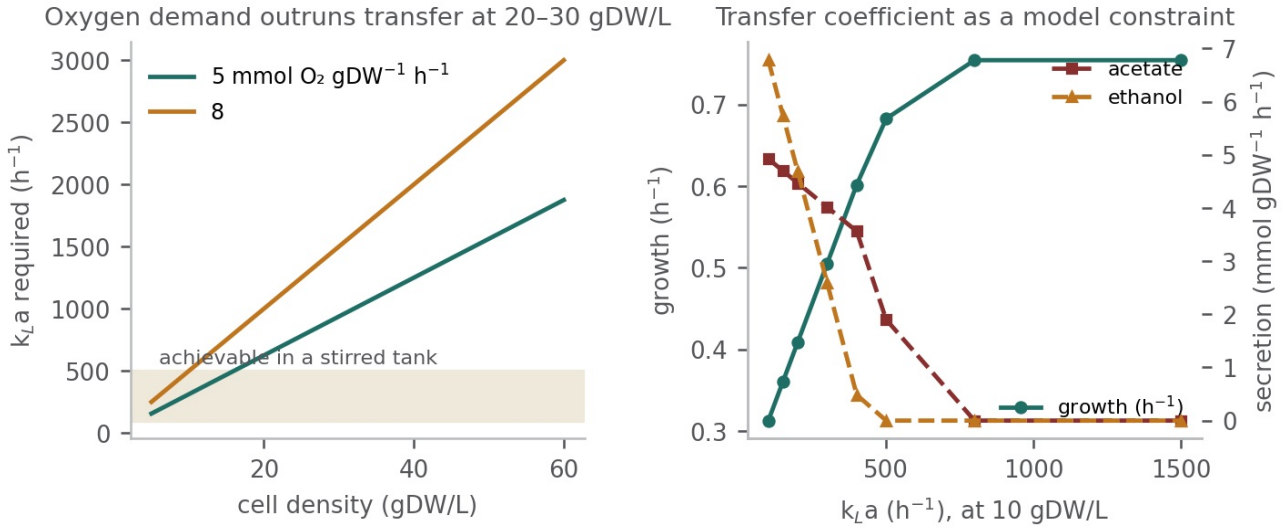


Figure 44.1 Left: the k_La required against cell density for two specific oxygen uptake rates, with the achievable range shaded — demand outruns transfer between 20 and 30 gDW/L. Right: growth and by-product secretion predicted by the core model when the oxygen exchange bound is set from the achievable transfer rate at 10 gDW/L. Computed for this module.

Read the right-hand panel as the answer to "what does my k_La cost me". At 100 h^{-1} the model loses 58% of its growth and excretes nearly 12 mmol of reduced by-products; at 500 it loses 9% and excretes only acetate; at 1,500 it reaches the unconstrained optimum. **The by-products are the diagnostic**: a fermentation secreting ethanol and formate aerobically is reporting an oxygen limitation, and the model quantifies how much.

What the cell actually experiences

Mixing time tens of seconds means a cell circulating through a large vessel passes through regions of different substrate and oxygen concentration on that timescale: high substrate near the feed point, oxygen-rich near the sparger, depleted elsewhere. Compare with the cell's own response times, which part 1 chapter 1 and part 2 chapter 16 put at seconds for metabolite pools, minutes for transcription and mRNA turnover, and a generation for protein dilution. So the cell is **fast enough to respond to a gradient it circulates through** — pools and fluxes adjust within a pass — but too slow to re-tune its proteome, and the result is repeated feast–famine cycling with a transcriptional stress response superimposed.

A **scale-down reactor** reproduces that in 1 L: two compartments, a stirred tank and a plug-flow section connected in a loop, with the residence time in the plug-flow leg set to the large vessel's circulation time, and the feed introduced at one point. The cells then experience the same feast–famine period in a vessel you can afford to run in replicate. What measurement would show it mattered: by-product formation and yield against the well-mixed control at identical average conditions, plus a stress reporter or transcript panel; if the two-compartment run secretes more acetate and yields less at the same average feed rate, the gradient is the cause of the scale-up loss.

Feeding strategy

The substrate balance for a fed-batch, with V the volume, F the feed rate and S_{feed} its concentration:

```
d(SV)/dt = F·S_feed − (μ/Y_XS + m)·X·V
```

To hold μ constant while biomass grows exponentially, the feed must grow at the same rate, which gives the **exponential feed** $F(t) = F_0 \cdot e^{\mu t}$. The consequence is the one worth stating in an interview: **the pump, not the organism, sets the growth rate**, and that is the point — it keeps the specific uptake rate below the overflow threshold of part 1 chapter 6 and the oxygen demand below the transfer limit above.

Strategy	What it does	Failure mode
Constant feed	Fixed F; μ falls as biomass accumulates	Early overflow, late starvation; simple and rarely optimal
Exponential	$F_0 e^{\mu t}$ holds μ at a set point	Open-loop: any deviation in yield or viability compounds, and the feed outruns the culture
DO-stat	Feed when dissolved oxygen rises above a threshold (substrate exhausted)	Oscillation; relies on the DO probe; confounded if oxygen transfer changes
pH-stat	Feed on the pH rise caused by substrate exhaustion	Only works when the organism's acid production tracks substrate; base addition confounds it

Choosing from physiology rather than convention: a Crabtree-positive yeast or an *E. coli* with a sharp overflow threshold needs the specific uptake rate held below that threshold, so exponential feeding at a μ below the critical value, with DO-stat as a safety net. An organism with no overflow metabolism — *C. glutamicum*, part 6 chapter 59 — tolerates a more generous feed and may be limited by oxygen instead. An organism making a growth-decoupled product (part 5 chapter 56's PHA) wants a two-phase feed: exponential during growth, then carbon-only with nitrogen withheld.

Downstream, and the cost sketch

The unit operations in order, with the property each exploits: **centrifugation or filtration** — density or size, to separate cells from broth; **homogenisation** — mechanical, if the product is intracellular; **clarification** — size again, to remove debris; **extraction or adsorption** — partitioning (part 3 chapters 45 and 46); **chromatography** — specific binding, for high-value products; **distillation** — volatility; **crystallisation** — solubility; **drying** — volatility again. Which of these you need is set by where the product is and what it is, which is part 6 chapter 59's localisation column.

Then the water, which is the number that decides the train:

Titer	Broth per kg of product	Water to remove
5 g/L	200 L	199 kg
20 g/L	50 L	49 kg
50 g/L	20 L	19 kg
100 g/L	10 L	9 kg

At 20 g/L you move fifty kilograms of water per kilogram of product, and evaporating it would take about 113 MJ — against the 20 to 30 MJ of chemical energy in a kilogram of most organic products. That is the part 6 chapter 60 dewatering argument in a fermenter, and it is why the cost sketch of part 4 chapter 49 is dominated by feedstock and separation, and why the titer below which a process cannot work is set by the separation rather than by the biology.

The back-of-envelope, with part 4 chapter 49's model: feedstock cost is substrate price divided by yield, so at $0.40/kg glucose and 0.40 g/g it is $1.00/kg of product at the theoretical yield and $1.60 at a realistic 0.25 g/g; capital and fixed costs scale roughly as 1/titer. Setting those against the product's market price gives the titer below which the process cannot work — which for a $1.80/kg commodity in that model was around 100 g/L.

Power	$P = N_p \rho n^3 d^5$; $N_p \approx 5$ for a Rushton turbine in turbulent flow
Constant P/V	$n \propto d^{(-2/3)}$; tip speed $\propto d^{(1/3)}$; mixing time $\propto d^{(2/3)}$
Transfer	$OTR = k_L a(C^* - C_L)$; $C^* \approx 0.21$ mM for air at 37 °C
The wall	50 gDW/L at 5 mmol gDW^{-1} h^{-1} needs $k_L a \approx 1{,}560$ h^{-1} against 100–500 achievable
As a model bound	OTR/X in mmol gDW^{-1} h^{-1} becomes the oxygen exchange bound
Exponential feed	$F(t) = F_0 e^{(\mu t)}$: the pump sets the growth rate
Water	1,000/titer litres of broth per kg; 50 kg of water per kg at 20 g/L

Your 1 L fermentation reaches 40 gDW/L and 25 g/L of product. At 50 m³ the same protocol gives 22 gDW/L, 11 g/L, and acetate in the broth. Diagnose it.

Oxygen first, because the numbers point there. At 40 gDW/L and 5 mmol·gDW^{-1}·h^{-1} the demand is 200 mmol·L^{-1}·h^{-1}, needing a k_La of 1,250 h^{-1}, which a 1 L vessel at high stirrer speed can approach and a 50 m³ vessel cannot — so the large culture is oxygen-limited, and the acetate is exactly what the core model predicts when the oxygen bound is tightened, as the table above shows. The biomass stopping at 22 gDW/L is consistent: growth is limited by the oxygen the vessel can deliver.

What to check to confirm it, in order: the dissolved-oxygen trace, which should be at or near zero during the productive phase; the off-gas, for an oxygen uptake rate that plateaus while the feed continues; and the acetate against the model's prediction at the measured OTR. Then the fixes, cheapest first — reduce the feed rate so the specific uptake rate falls below what the vessel can support, which costs time; enrich the inlet oxygen or raise the head pressure, which costs utilities; or accept a lower cell density and raise the specific productivity instead. And the thing not to do is to assume the strain degenerated: the gradient and transfer explanation is testable in a scale-down reactor before any strain work begins.

Three things people get wrong

- ‘Hold power per volume constant and the scale-up is matched.’ Mixing time still grows as $d^{2/3}$ — 13-fold from 1 L to 100 m³.
- ‘Add more air.’ k_La rises sublinearly with power and gas flow; at 50 gDW/L you need 1,560 h^{-1}, which air at atmospheric pressure cannot give.
- ‘The strain degenerated at scale.’ Check the oxygen and the gradients first — both are testable in a 1 L scale-down vessel.

Self-check — answer on paper first

1. Derive why mixing time grows with scale at constant power per volume.

Power is $P = N_p \rho n^3 d^5$ and, under geometric similarity, volume scales as d^3, so $P/V \propto n^3 d^2$. Holding that constant requires $n \propto d^{-2/3}$. Mixing time for a given geometry scales roughly as $1/n$, so it scales as $d^{2/3}$ — a 45-fold increase in impeller diameter, from 4 cm to 1.8 m, gives a mixing time about 12.7 times longer. Tip speed $nd \propto d^{1/3}$ rises at the same time, so the larger vessel is simultaneously worse mixed and more shear-intense at the impeller.

2. A culture needs 150 mmol·L^{-1}·h^{-1} of oxygen and your vessel manages k_La = 400 h^{-1}. What are the options, quantitatively?

With $C^* = 0.21$ and $C_L = 0.05$ mM the available OTR is $400 \times 0.16 = 64$ mmol·L^{-1}·h^{-1}, well short of 150. Enriching the inlet gas to 50% oxygen raises C^* to about 0.5 mM, giving a driving force of 0.45 and an OTR of 180 — sufficient. Raising the head pressure to 2 bar absolute roughly doubles C^*, giving about 134 — nearly sufficient, and combinable with modest enrichment. Raising power would need k_La up by a factor of 2.3, which at an exponent near 0.5 on P/V means about five times the power, with the shear and cost that implies. Or reduce the demand: feed more slowly, which lowers the specific uptake rate and is usually the cheapest answer.

3. Why does an exponential feed hold the growth rate constant, and what is its failure mode?

Because at quasi-steady state the substrate supplied per unit time must match consumption, which is $(\mu/Y_{XS} + m) \cdot X \cdot V$; with X growing as $e^{\mu t}$, matching requires the feed to grow at the same exponential rate, so $F(t) = F_0 e^{\mu t}$ fixes μ at the chosen value. The failure mode is that it is open-loop: it assumes the yield, the maintenance coefficient and the viability stay as designed, and if the culture underperforms — a lower yield, a stress response, partial loss of viability — the feed keeps rising on schedule while consumption does not, substrate accumulates, and the culture tips into overflow metabolism or substrate inhibition. Hence a DO or substrate measurement as a safety net.

What to open a book for. Doran's *Bioprocess Engineering Principles*, mass-transfer and reactor chapters, worked rather than read — this week is the one where working the problems is the content. Shuler, Kargi & DeLisa on reactor operation and scale-up, Blanch & Clark for depth, a scale-up and scale-down review for the gradient material, and your own transport phenomena notes, which cover most of the theory already.

Closing notes to part 11

The six weeks in one page

Week 39. Growth coupling is the lower edge of the production envelope. An enumerated search found no coupled design aerobically and 52 coupled pairs anaerobically; ΔPYK gives 8.23 mmol succinate at μ 0.134, the triple 9.36 at 0.048 — more coupling, less growth. Capping the product export at 6 kills growth and below 4 there is no feasible solution at all. A crude proteome budget reproduces overflow and falling oxygen uptake.

Week 40. Rule-based expansion is combinatorial, so ranking matters more than generation, and the ranking that matters is enzyme availability and length: a route of n steps each with probability p works with probability p^n — 33% at five steps and p = 0.8. A step with no candidate enzyme is a discovery project, not a low probability.

Week 41. Bayesian optimisation reached 98% of the achievable titer in eleven runs where a 16-run factorial grid reached 78% and never more, across forty seeds — and one seed in forty stuck at 69% on a local optimum. Report the spread.

Week 42. Twenty samples and sixty features gave a training R^2 of 1.00 and a held-out R^2 of 0.19; with shuffled labels, 1.00 and −1.67. The same family-structured dataset scored 0.67 under random cross-validation and −0.44 when grouped by family. The split must match what you need to predict.

Week 43. [1-^{13}C] glucose reports the glucose-6-phosphate split because carbon 1 is the carbon the oxidative branch decarboxylates; fitting a split from MIDs with 0.4% noise gave 0.298 with a bootstrap interval of 0.289–0.312. Natural abundance alone gives 3.2% M+1 for a three-carbon fragment. A narrow interval on a badly specified network is worth nothing.

Week 44. At constant power per volume, mixing time grows as $d^{2/3}$ — 12.7-fold from 1 L to 100 m³. A culture at 50 gDW/L needs a k_La of 1,560 h^{-1} against 100–500 achievable. Feeding that transfer limit into the core model as an oxygen bound: at k_La 100 the model loses 58% of its growth and excretes 12 mmol of reduced by-products; at 1,500 it reaches the unconstrained optimum.

On the numbers in this part

The strain-design search, the proteome-constrained sweep, the Bayesian optimisation comparison, the overfitting and leakage simulations, the ^{13}C fit and the oxygen-bound sweep were all executed. Their parameters are the assumptions: the core model of part 10 with its P/O ratios and rounded biomass coefficients; enzyme costs in the proteome constraint that are illustrative rather than measured, so the onset of overflow is the result and the behaviour at high uptake is an artefact of a hard budget; a response surface, noise level and kernel in chapter 41 that I chose; a feature count and signal structure in chapter 42 that set how fast the learning curve flattens; a deliberately minimal atom-transition model in chapter 43, where a real analysis uses the full network and software I could not run; and in chapter 44 a Rushton power number, a specific oxygen uptake of 5 $mmol \cdot gDW^{-1} \cdot h^{-1}$, and C* = 0.21 mM.

Not run, and marked where they appear: RetroPath2.0 and any retrosynthesis tool, the ^{13}C fitting packages, and COBRApy. The two results I would most want checked against real data are the proteome cost vector, because it decides where overflow begins, and the specific oxygen uptake rate, because every k_La number in chapter 44 scales with it.

What remains

Written so far: weeks 1 to 44 and 45 to 61 — 61 of the tracker's 66. What is left is weeks 62 to 66: how publishing works, writing a proposal, the application and interview phase, and the capstone. They are about how the profession works rather than about metabolic engineering, and as I said when part 6 went out, they are the ones nobody writes down for an engineer arriving from another discipline. That is the last part.